Lecture Notes for

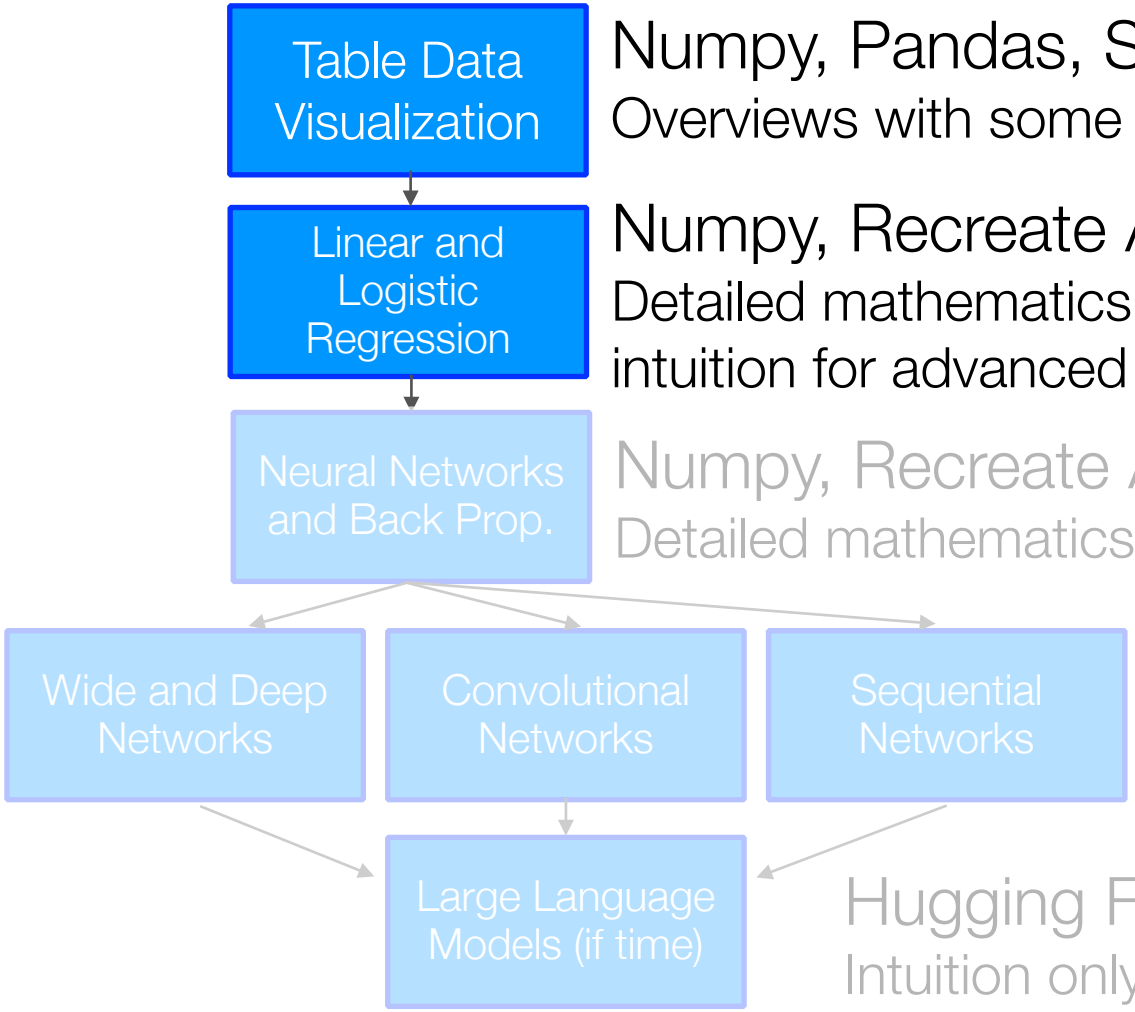
Neural Networks I

Logistic Regression I & II

Class Logistics and Agenda

- Last Time: Linear Regression
- Logistics
  - Grading
  - TA Office Hours Posted
  - Final Exam Dates
- Agenda
  - Logistic Regression
    - Solving and Programming

Class Overview, by topic



Numpy, Pandas, Seaborn
Overviews with some in-depth discussion

Numpy, Recreate API for Scikit-learn
Detailed mathematics for simple optimization
intuition for advanced optimization

Numpy, Recreate API for DL Frameworks
Detailed mathematics for NN operations

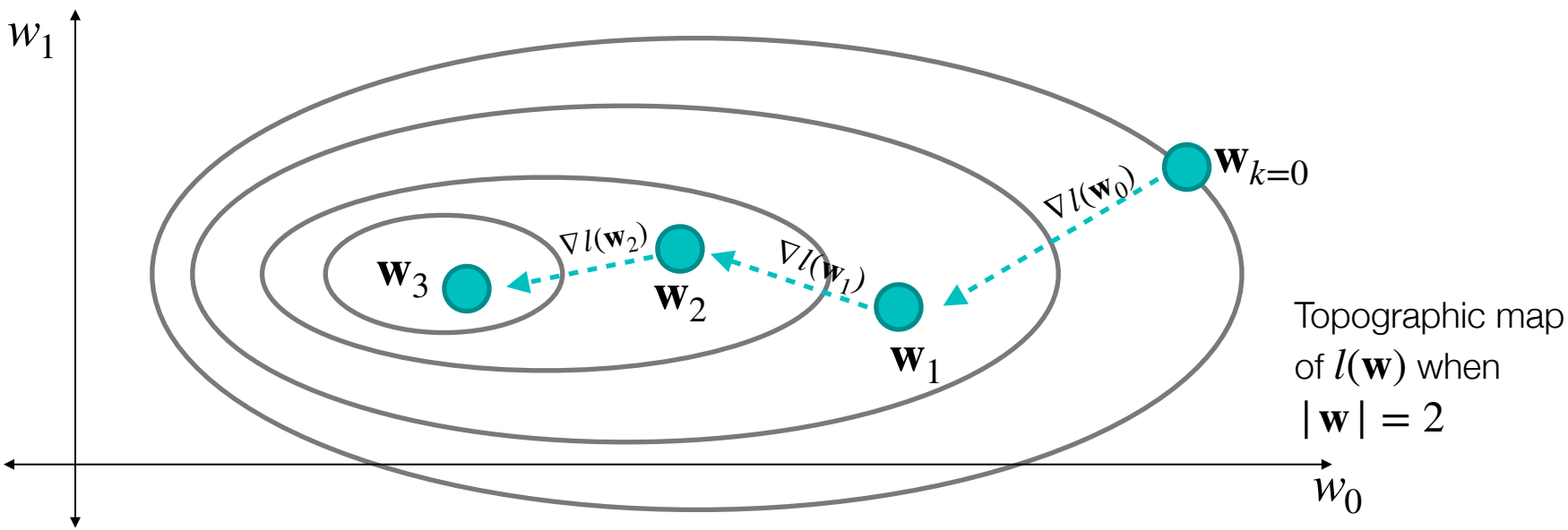
Hugging Face
Intuition only

Optimizing a function of multiple variables

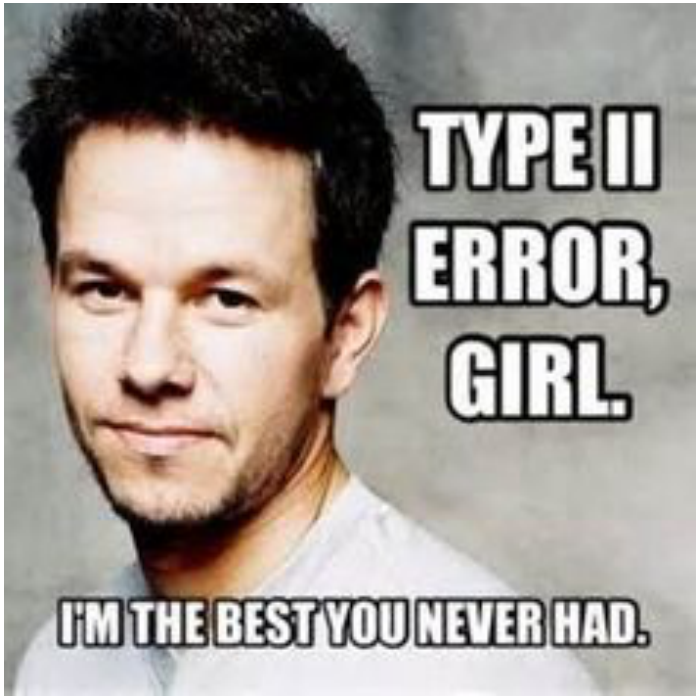
- Given an Objective function: $L(\mathbf{w})$ or $l(\mathbf{w})$
- With parameters: θ or $\mathbf{w}$ or $\mathbf{b}$ or $\mathbf{W}$ (the variables)
- Choose random starting point $\mathbf{w}_0$
- Need to calculate a Direction to move parameters:

$$\nabla L(\mathbf{w}) \text{ or } \nabla l(\mathbf{w}) = \frac{\partial}{\partial \mathbf{w}} l(\mathbf{w}) \text{ (the gradient)}$$

- Use direction in an update rule: $\mathbf{w}_{k+1} \leftarrow \mathbf{w}_k + \eta \cdot \nabla l(\mathbf{w})$



Logistic Regression

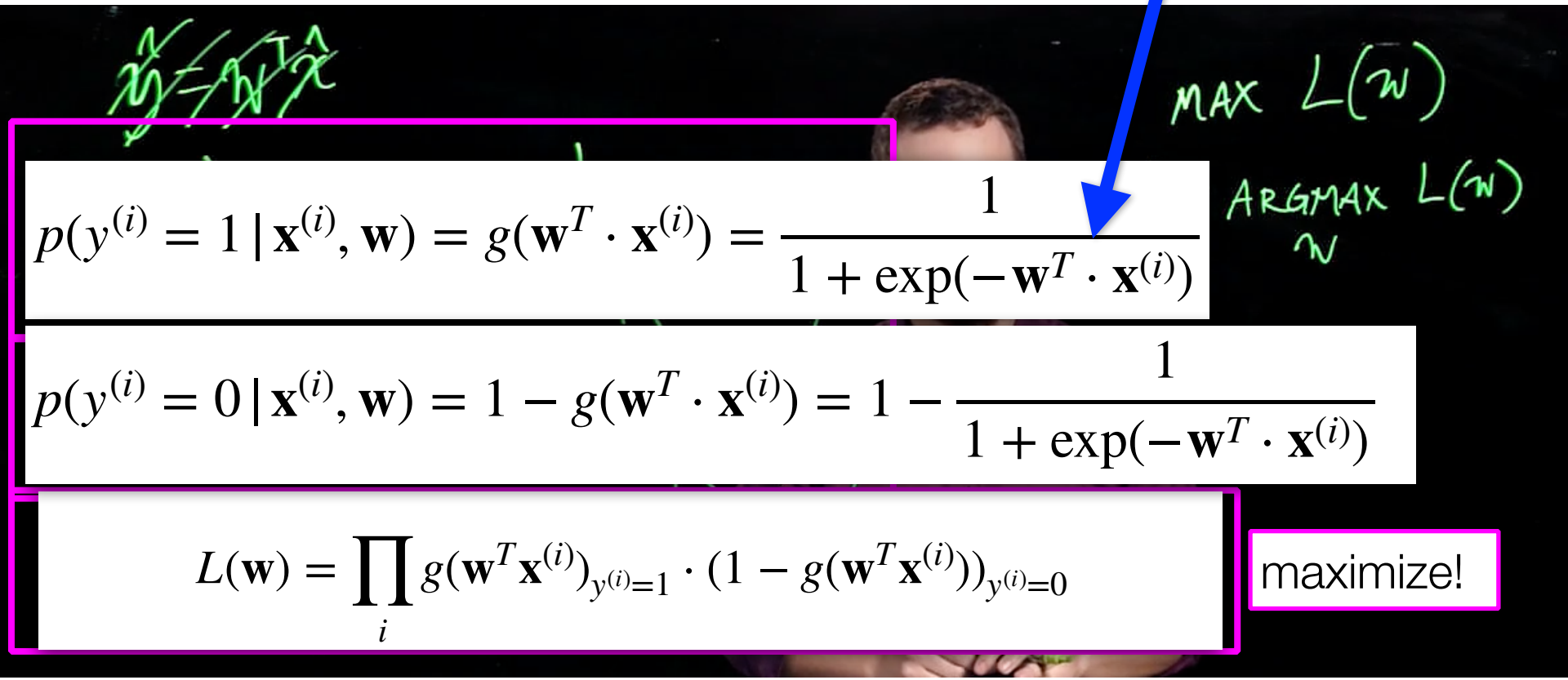


@researchmark

Setting Up Binary Logistic Regression

- From flipped lecture:

This notation assumes that $\mathbf{x}^{(i)}$ is a *column* vector, not *row*…



$$p(y^{(i)} = 1 \,|\, \mathbf{x}^{(i)}, \mathbf{w}) = g(\mathbf{w}^T \cdot \mathbf{x}^{(i)}) = \frac{1}{1 + \exp(-\mathbf{w}^T \cdot \mathbf{x}^{(i)})}$$

$$p(y^{(i)} = 0 \,|\, \mathbf{x}^{(i)}, \mathbf{w}) = 1 - g(\mathbf{w}^T \cdot \mathbf{x}^{(i)}) = 1 - \frac{1}{1 + \exp(-\mathbf{w}^T \cdot \mathbf{x}^{(i)})}$$

$$L(\mathbf{w}) = \prod_i g(\mathbf{w}^T \mathbf{x}^{(i)})_{y^{(i)}=1} \cdot (1 - g(\mathbf{w}^T \mathbf{x}^{(i)}))_{y^{(i)}=0}$$

maximize!

where $g(.)$ is a sigmoid and probability that $y^{(i)} = 1$

Expanded Notation, Linear Algebra

$$p(y^{(i)} = 1 \,|\, \mathbf{x}^{(i)}, \mathbf{w}) = g(\mathbf{x}^{(i)} \cdot \mathbf{w}) = \frac{1}{1 + \exp(-\mathbf{x}^{(i)} \cdot \mathbf{w})}$$

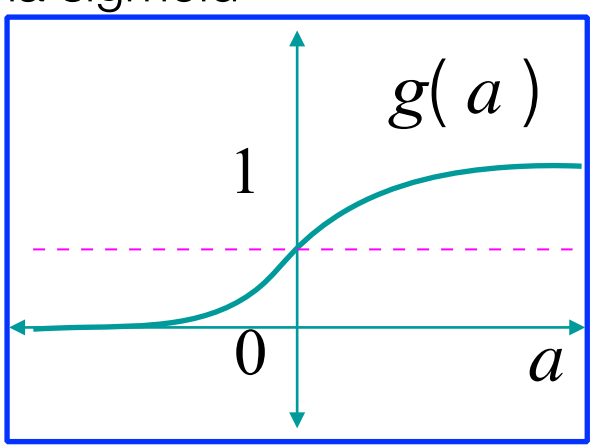
$$\mathbf{X} = \begin{bmatrix} 1 & x_1^{(1)} & x_2^{(1)} & \dots & x_N^{(1)} \\ 1 & x_1^{(2)} & x_2^{(2)} & \dots & x_N^{(2)} \\ & \vdots & \vdots & \ddots & \vdots \\ 1 & x_1^{(M)} & x_2^{(M)} & \dots & x_N^{(M)} \end{bmatrix} = \begin{bmatrix} \leftarrow & \mathbf{x}^{(1)} & \rightarrow \\ \leftarrow & \mathbf{x}^{(2)} & \rightarrow \\ & \vdots & \\ \leftarrow & \mathbf{x}^{(M)} & \rightarrow \end{bmatrix}$$

table data with ones
$M \times (N+1)$

projections onto $\mathbf{w}$

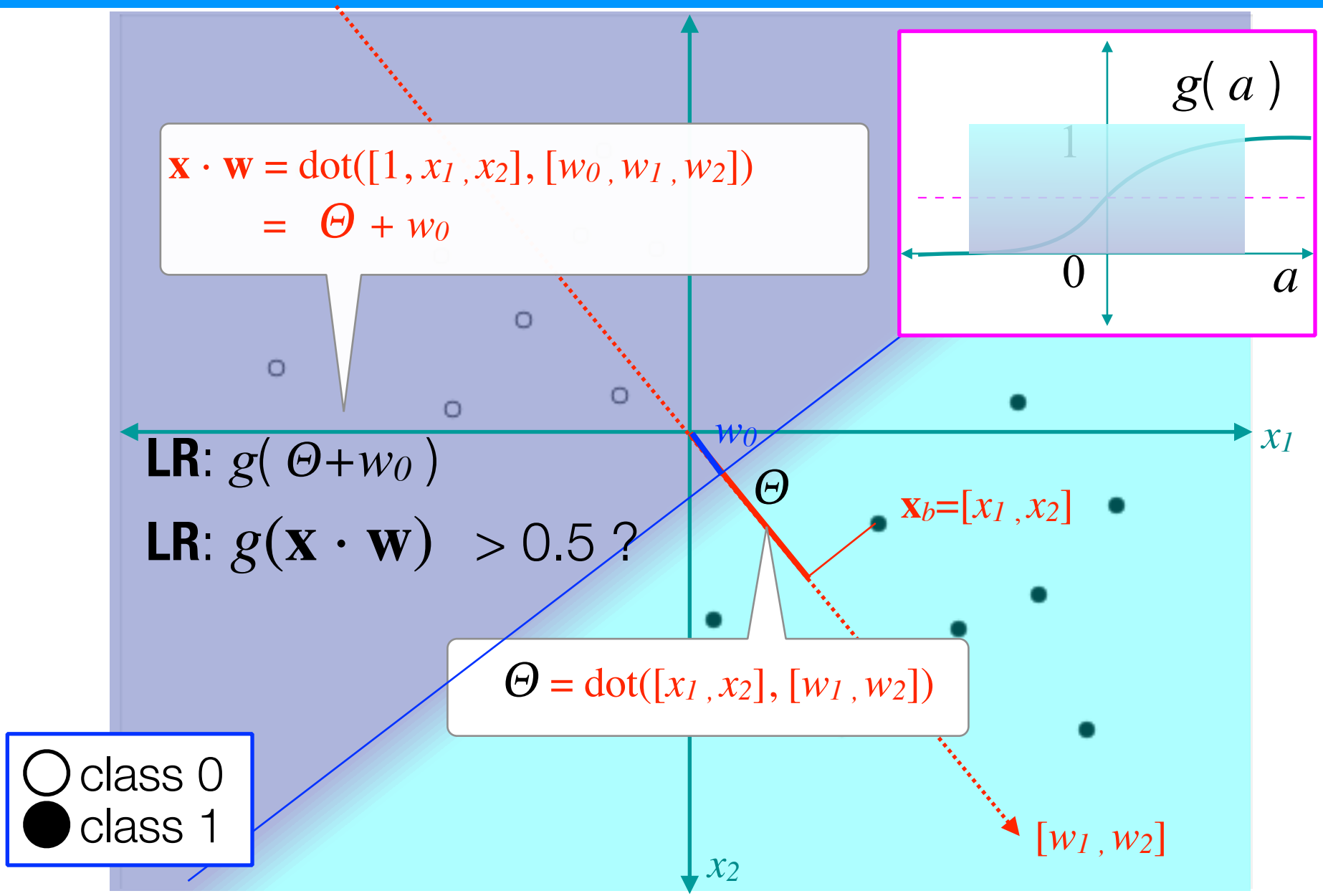
$$\mathbf{X} \cdot \mathbf{w} = \begin{bmatrix} \leftarrow & \mathbf{x}^{(1)} & \rightarrow \\ \leftarrow & \mathbf{x}^{(2)} & \rightarrow \\ & \vdots & \\ \leftarrow & \mathbf{x}^{(M)} & \rightarrow \end{bmatrix} \cdot \begin{bmatrix} w_0 \\ w_1 \\ \vdots \\ w_N \end{bmatrix} = \begin{bmatrix} \mathbf{x}^{(1)} \cdot \mathbf{w} \\ \mathbf{x}^{(2)} \cdot \mathbf{w} \\ \vdots \\ \mathbf{x}^{(M)} \cdot \mathbf{w} \end{bmatrix}$$

map projection to probability via sigmoid

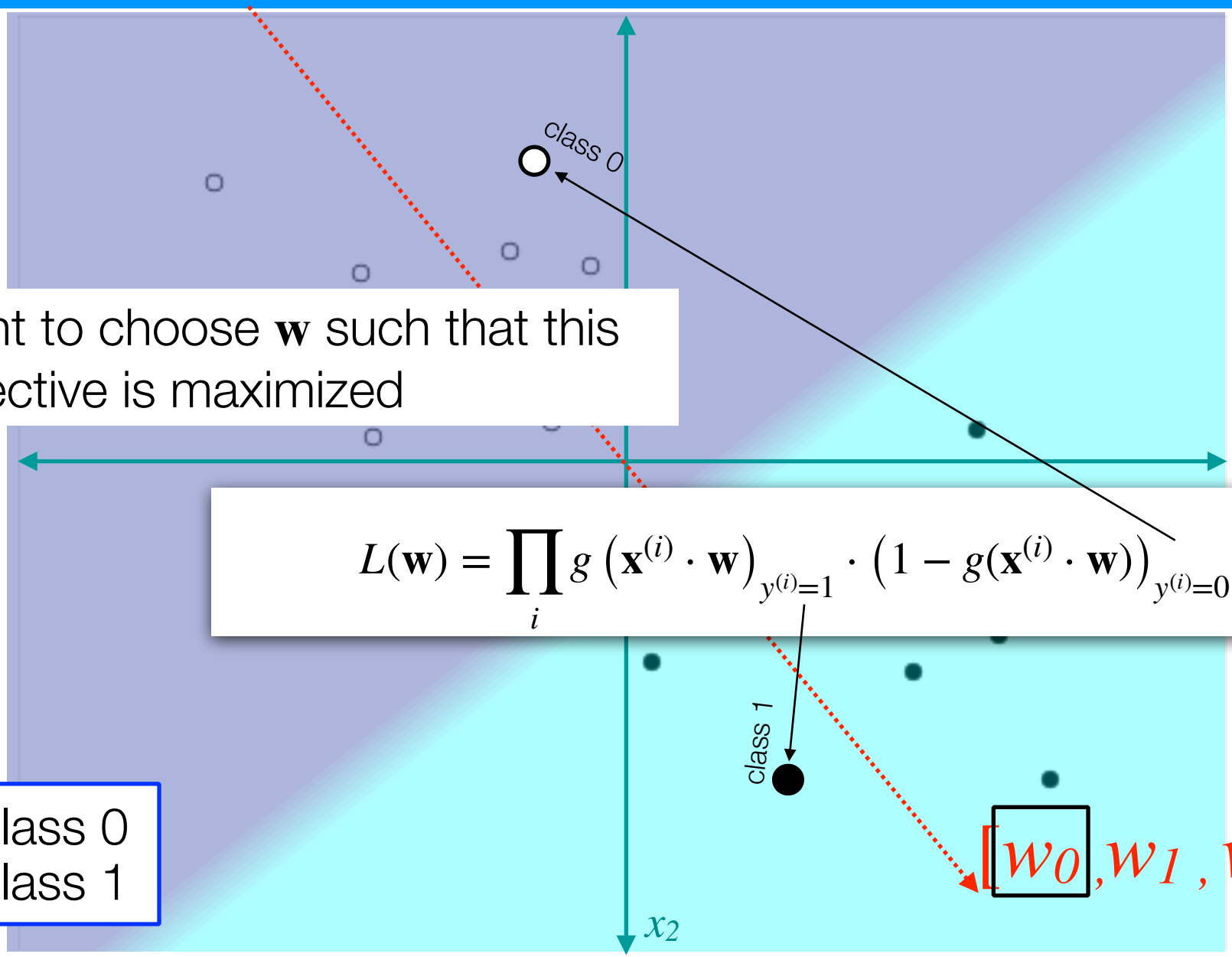


$$g\,(\mathbf{X} \cdot \mathbf{w}) = \begin{bmatrix} g(\mathbf{x}^{(1)} \cdot \mathbf{w}) \\ g(\mathbf{x}^{(2)} \cdot \mathbf{w}) \\ \vdots \\ g(\mathbf{x}^{(M)} \cdot \mathbf{w}) \end{bmatrix} = \begin{bmatrix} p(y^{(1)} = 1 \,|\, \mathbf{x}^{(1)}) \\ p(y^{(2)} = 1 \,|\, \mathbf{x}^{(2)}) \\ \vdots \\ p(y^{(M)} = 1 \,|\, \mathbf{x}^{(M)}) \end{bmatrix}$$

$\mathbf{w} = [w_0, w_1, w_2]$, Geometric Interpretation



Changing w alters probability



Want to choose $\mathbf{w}$ such that this objective is maximized

$$L(\mathbf{w}) = \prod_i g\left(\mathbf{x}^{(i)} \cdot \mathbf{w}\right)_{y^{(i)}=1} \cdot \left(1 - g(\mathbf{x}^{(i)} \cdot \mathbf{w})\right)_{y^{(i)}=0}$$

class 0
class 1

Optimize iteratively

- **Objective Function**: likelihood, given below
- **Parameters**: changing $\mathbf{w}$
- **Update Direction**: need to calculate gradient to know how to update initial guess of $\mathbf{w}$

$$L(\mathbf{w}) = \prod_i g\left(\mathbf{x}^{(i)} \cdot \mathbf{w}\right)_{y^{(i)}=1} \cdot \left(1 - g(\mathbf{x}^{(i)} \cdot \mathbf{w})\right)_{y^{(i)}=0}$$

- Simplify $L(\mathbf{w})$ with **logarithm**, $l(\mathbf{w})$

$$l(\mathbf{w}) = \sum_i y^{(i)} \ln\left(g(\mathbf{x}^{(i)} \cdot \mathbf{w})\right) + (1 - y^{(i)}) \ln\left(1 - g(\mathbf{x}^{(i)} \cdot \mathbf{w})\right)$$

Logistic Regression Optimization Procedure

Log Likelihood

$$l(\mathbf{w}) = \sum_i y^{(i)} \ln\left(g(\mathbf{x}^{(i)} \cdot \mathbf{w})\right) + (1 - y^{(i)}) \ln\left(1 - g(\mathbf{x}^{(i)} \cdot \mathbf{w})\right)$$

- **Take** Gradient

$$\frac{\partial}{\partial w_j} l(\mathbf{w}) = \sum_i \left(y^{(i)} - g(\mathbf{x}^{(i)} \cdot \mathbf{w})\right) x_j^{(i)}$$

- **Use** gradient to **update** equation for $\mathbf{w}$

$$\mathbf{w}_{k+1} \leftarrow \mathbf{w}_k + \eta \cdot \nabla l(\mathbf{w})$$

- Video Supplement (also on canvas):
  - https://www.youtube.com/watch?v=FGnoHdjFrJ8

Binary Solution for Update Equation

- Use gradient inside update equation for **w**

$$\frac{\partial}{\partial w_j} l(\mathbf{w}) = \sum_i \left(y^{(i)} - g(\mathbf{x}^{(i)} \cdot \mathbf{w})\right) x_j^{(i)}$$

$$\mathbf{w}_{k+1} \leftarrow \mathbf{w}_k + \eta \cdot \nabla l(\mathbf{w})$$

$$\underbrace{w_j}_{\text{new value}} \leftarrow \underbrace{w_j}_{\text{old value}} + \underbrace{\frac{\eta}{M}}_{\text{step}} \underbrace{\sum_{i=1}^{M} (y^{(i)} - g(\mathbf{x}^{(i)} \cdot \mathbf{w}))x_j^{(i)}}_{\text{gradient } \nabla l(\mathbf{w})}$$

This updates each element of gradient, how to calculate for all elements of the gradient update?

$$\mathbf{w} \leftarrow \mathbf{w} + \frac{\eta}{M} \sum_{i=1}^{M} (y^{(i)} - g(\mathbf{x}^{(i)} \cdot \mathbf{w})) \cdot \mathbf{x}^{(i)}$$

Reinvent sklearn Logistic Regression **Demo**

03. Logistic Regression.ipynb

Programming

Vectorization

Regularization

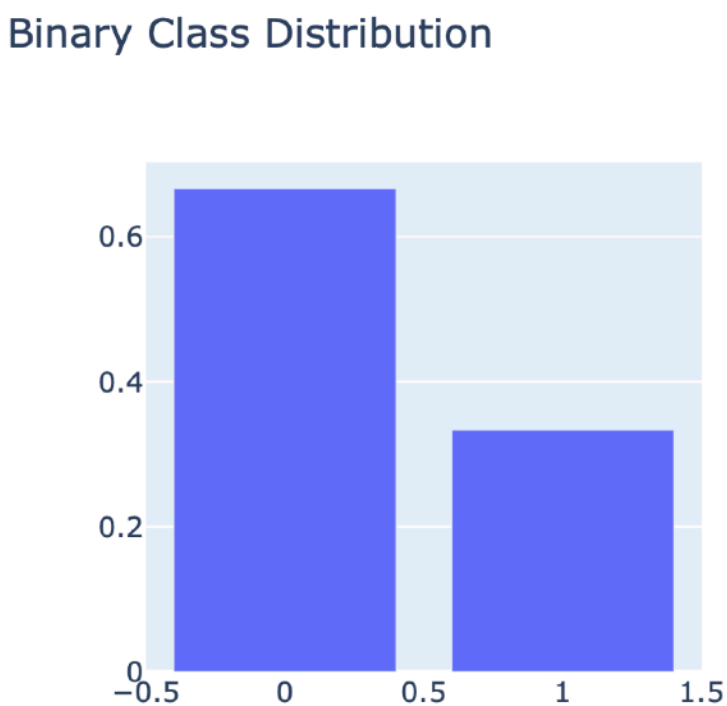
Multi-class extension

```
>>> from sklearn.datasets import load_iris
>>> from sklearn.linear_model import LogisticRegression
>>> X, y = load_iris(return_X_y=True)
>>> clf = LogisticRegression(random_state=0).fit(X, y)
>>> clf.predict(X[:2, :])
array([0, 0])
>>> clf.predict_proba(X[:2, :])
array([[9.82e-01, 1.82e-02, 1.44e-08],
       [9.72e-01, 2.82e-02, 3.02e-08]])
>>> clf.score(X, y)
0.97
```

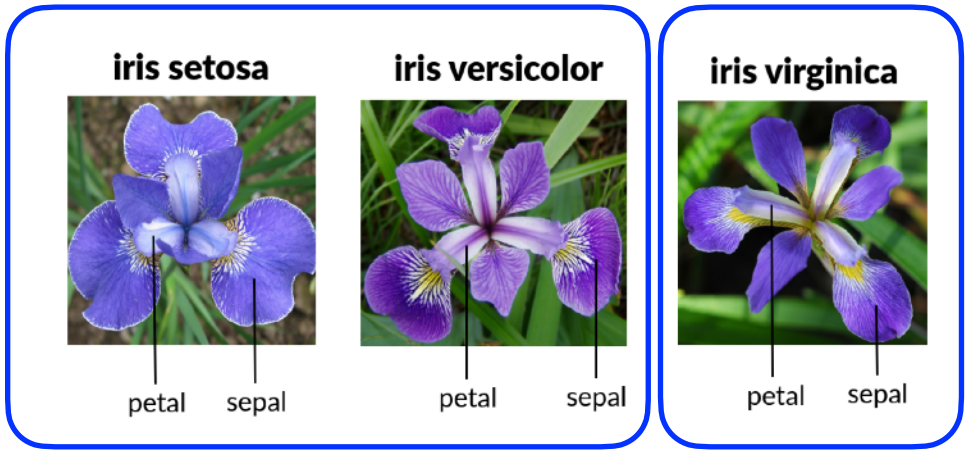
Jupyter notebook demo is available, but all code is truncated and included in these slides…

Dataset for Testing Binary Logistic Regression

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split

ds = load_iris()
X = ds.data
y = (ds.target>1).astype(int) # make problem binary

X_train, X_test, y_train, y_test = train_test_split(X,y, train_size = 0.8, test_size=0.2)
```



Load iris dataset,
four features: width, length or petal/sepal
make two classes (0 or 1)



Class 0 Class 1

Base Code for Logistic Regression

```
class BinaryLogisticRegressionBase:
   def __init__(self, eta, iterations=20):
        self.eta = eta
        self.iters = iterations

    def __str__(self):
        return 'Base Binary Logistic Regression Object, Not Trainable'

    @staticmethod
    def _sigmoid(theta):
        return 1/(1+np.exp(-theta))

    @staticmethod
    def _add_intercept(X):
        return np.hstack((np.ones((X.shape[0],1)),X)) # add bias term

    @property
    def coef_(self):
        return self.w_[1:]


    @property
    def intercept_(self):
        return self.w_[0]


    def predict_proba(self, X, add_intercept=True):
        Xb = self._add_intercept(X) if add_intercept else X # add bias term if requested
        return self._sigmoid(Xb @ self.w_) # return the probability y=1

    def predict(self,X):
        return (self.predict_proba(X)>0.5) #return the actual prediction
```

Define same style interface as `sklearn`.

After "`fit`" we set various properties

Fitting Logistic Regression (slow version)

```
class BinaryLogisticRegression(BinaryLogisticRegressionBase):
    def __str__(self):
        if(hasattr(self,'w_')): return 'Binary Logistic Regression coefficients:\n'+ str(self.w_)
        else: return 'Untrained Binary Logistic Regression Object'

    def _get_gradient(self,X,y):
        gradient = np.zeros(self.w_.shape)
        for (xi,yi) in zip(X,y):
            gradi = (yi - self.predict_proba(xi,add_intercept=False))*xi
            gradient += gradi.reshape(self.w_.shape)
        return gradient/float(len(y))

    def fit(self, X, y):
        Xb = self._add_intercept(X) # add bias term
        num_samples, num_features = Xb.shape
        self.w_ = np.zeros((num_features,1)) # init weight vector to zeros

        for _ in range(self.iters):
            gradient = self._get_gradient(Xb,y)
            self.w_ += gradient*self.eta # multiply by learning rate
```

$$\nabla l(\mathbf{w}) = \frac{1}{M}\sum_{i=1}^{M}(y^{(i)} - g(\mathbf{x}^{(i)}\cdot\mathbf{w}))\cdot\mathbf{x}^{(i)}$$

$$\mathbf{w}_{init} = [0...0]$$

$$\mathbf{w} \leftarrow \mathbf{w} + \eta\cdot\nabla l(\mathbf{w})$$

```
%%time
# Now we can train the classifier
blr = BinaryLogisticRegression(eta=0.1,iterations=12)
blr.fit(X_train,y_train)
print(blr)
```

```
Binary Logistic Regression Object with coefficients:
[[-0.0722316 ]
 [-0.18542668]
 [-0.24636172]
 [ 0.31415213]
 [ 0.19209272]]
CPU times: user 17.5 ms, sys: 2.59 ms, total: 20.1 ms
Wall time: 19.9 ms
```

```
print("coef",blr.coef_)
print("intercept",blr.intercept_)
print("w",blr.w_)
```

```
coef [[-0.18542668]
 [-0.24636172]
 [ 0.31415213]
 [ 0.19209272]]
intercept [-0.0722316]
w [[-0.0722316 ]
 [-0.18542668]
 [-0.24636172]
 [ 0.31415213]
 [ 0.19209272]]
```

```
from sklearn.metrics import accuracy_score

yhat = blr.predict(X_test)
print('Accuracy of: ',accuracy_score(y_test,yhat))
```

```
Accuracy of:  0.9
```

Fitting Logistic Regression (fast version)

```
def _get_gradient(self,X,y):
    gradient = np.zeros(self.w_.shape)
    for (xi,yi) in zip(X,y):
        gradi = (yi - self.predict_proba(xi,…))*xi
        gradient += gradi.reshape(self.w_.shape)
    return gradient/float(len(y))
```

prev `get_gradient`

$$\nabla l(\mathbf{w}) = \frac{1}{M}\boxed{\sum_{i=1}^{M}(y^{(i)} - g(\mathbf{x}^{(i)}\cdot\mathbf{w}))\cdot\mathbf{x}^{(i)}}$$

```
Binary Logistic Regression Object with coefficients:
[[-0.1864301 ]
 [-0.45680892]
 [-0.55976024]
 [ 0.79216699]
 [ 0.52832691]]
Accuracy of:  0.9
CPU times: user 267 ms, sys: 3.45 ms, total: 271 ms
Wall time: 270 ms
```

`for` loop: 270ms

$$\mathbf{y}_{diff} = \begin{bmatrix} y^{(1)} - g(\mathbf{x}^{(1)}\cdot\mathbf{w}) \\ y^{(2)} - g(\mathbf{x}^{(2)}\cdot\mathbf{w}) \\ \vdots \\ y^{(M)} - g(\mathbf{x}^{(M)}\cdot\mathbf{w}) \end{bmatrix} = \begin{bmatrix} y^{(1)}_{diff} \\ y^{(2)}_{diff} \\ \vdots \\ y^{(M)}_{diff} \end{bmatrix}$$

$$\mathbf{y}_{diff}\odot\mathbf{X} = \begin{bmatrix} y^{(1)}_{diff} \\ y^{(2)}_{diff} \\ \vdots \\ y^{(M)}_{diff} \end{bmatrix} \cdot \begin{bmatrix} \leftarrow & \mathbf{x}^{(1)} & \rightarrow \\ \leftarrow & \mathbf{x}^{(2)} & \rightarrow \\ & \vdots & \\ \leftarrow & \mathbf{x}^{(M)} & \rightarrow \end{bmatrix} = \begin{bmatrix} (\leftarrow & \mathbf{x}^{(1)} & \rightarrow)\cdot y^{(1)}_{diff} \\ (\leftarrow & \mathbf{x}^{(2)} & \rightarrow)\cdot y^{(2)}_{diff} \\ & \vdots & \\ (\leftarrow & \mathbf{x}^{(M)} & \rightarrow)\cdot y^{(M)}_{diff} \end{bmatrix}$$

```
[[-0.1864301 ]
 [-0.45680892]
 [-0.55976024]
 [ 0.79216699]
 [ 0.52832691]]
Accuracy of:  0.9
CPU times: user 16.6 ms, sys: 2.31 ms, total: 19 ms
Wall time: 17.6 ms
```

Vectorized: 17ms

$$\nabla l(\mathbf{w}) = \frac{1}{M}\sum_{i=1}^{M} y^{(i)}_{diff}\cdot\mathbf{x}^{(i)}$$

$$\nabla l(\mathbf{w}) = \text{mean}(\mathbf{y}_{diff}\odot\mathbf{X})_{columns}$$

```
class VectorBinaryLogisticRegression( BinaryLogisticRegression ):

    def _get_gradient(self,X,y):
        ydiff = y-self.predict_proba(X, add_intercept=False)
        gradient = np.mean(X * ydiff[:,np.newaxis], axis=0)
        return gradient
```

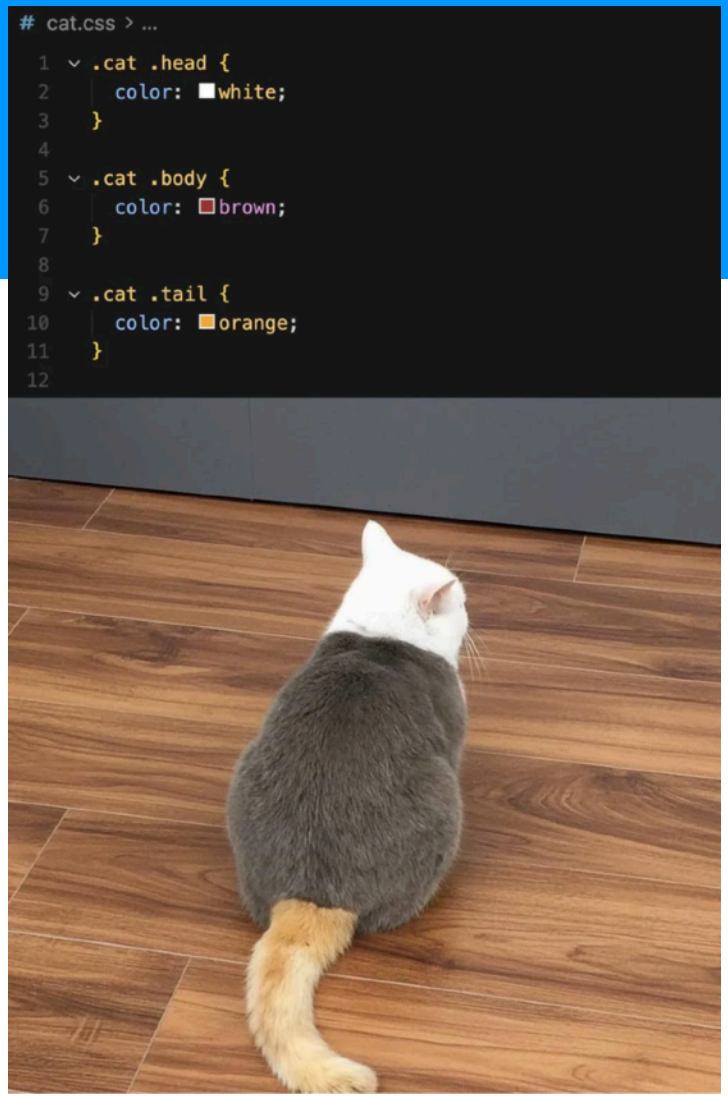
new `get_gradient`

Self Test

- 1. What part of the equation is represented by
  - `self.predict_proba(xi)`
  - $$\mathbf{w} \leftarrow \mathbf{w} + \eta \frac{1}{M} \sum_{i=1}^{M} \underbrace{(\underbrace{y^{(i)}}_{\mathbf{A}} - \underbrace{g(\mathbf{x}^{(i)} \cdot \mathbf{w})}_{\mathbf{B}})}_{\mathbf{D}} \underbrace{\mathbf{x}^{(i)}}_{\mathbf{C}}$$

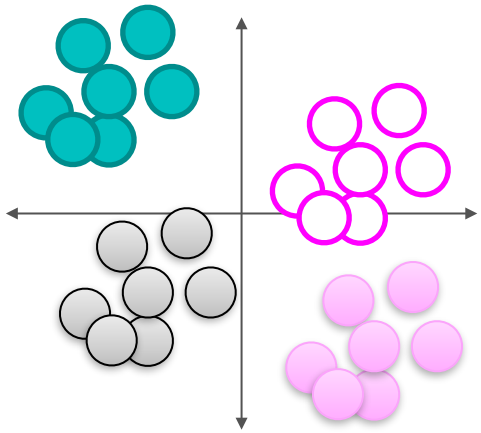
- 2. If we had a class imbalance problem, with one type of class dominating the response, how could we manipulate `fit` to give more importance to a given class?
  - A. Add weights to the gradient calculation (for each instance of `ydiff`)
  - B. Repeat certain classes in the update of the gradient calculation
  - C. Update `get_gradient` function to only update the worst performing class

Multi-class Logistic Regression



One Versus Rest

Multi-class formulation



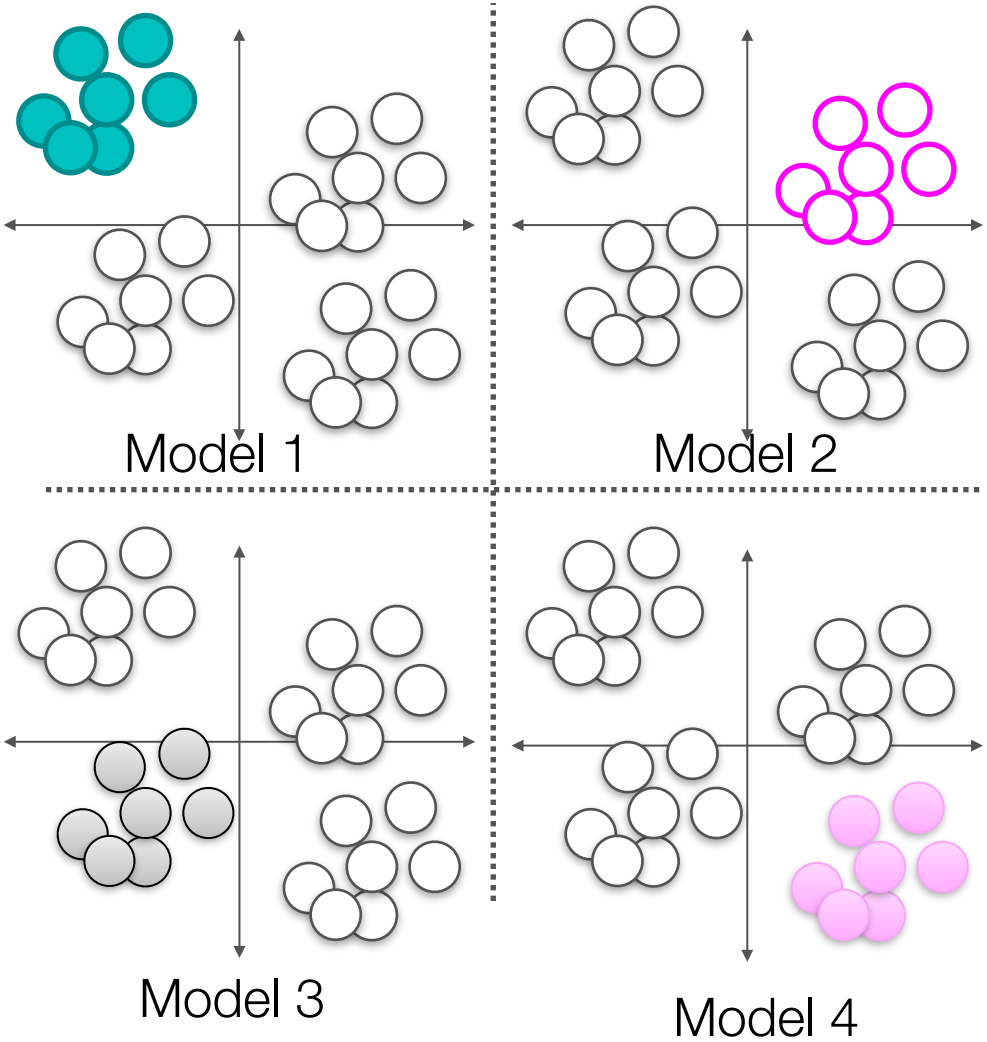
classes[0 1 2 3]

model probabilities

test data (rows)

M1	M2	M3	M4	arg max	hash
0.91	0.0	0.78	0.53	0	
0.53	0.5	0.32	**0.91**	3	
0.01	0.6	**0.98**	0.01	2	
0.05	**0.83**	0.21	0.81	1	
0.85	0.02	0.33	0.02	0	
0.63	0.11	0.09	0.33	0	
0.01	0.32	0.12	**0.88**	3	

Binary formulations

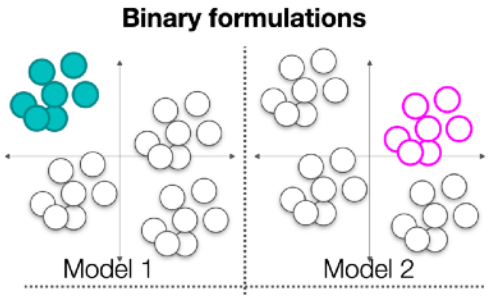


Programming OVR Logistic Regression

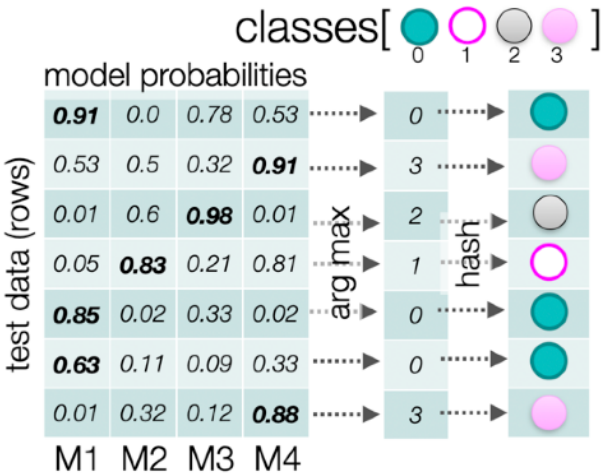
```
class LogisticRegression:
    def __init__(self, eta, iterations=20): ...
    def __str__(self): ...
    @property def coef_(self): ...
    @property def intercept_(self): ...
```

Class setup, interface like `sklearn`

```
    def fit(self,X,y):
        self.unique_ = np.unique(y) # list of unique class values
        num_unique_classes = len(self.unique_)
        self.classifiers_ = [] # fill array with binary classifiers

        for i,yval in enumerate(self.unique_): # for each unique value
            y_binary = (y==yval) # create a binary problem
            blr = VectorBinaryLogisticRegression(self.eta, self.iters)
            blr.fit(X,y_binary)
            self.classifiers_.append(blr)
```




```
    def predict_proba(self,X):
        probs = []
        for blr in self.classifiers_:
            probs.append(blr.predict_proba(X)) # per classifier

        return np.hstack(probs) # make into single matrix

    def predict(self,X):
        idx = np.argmax(self.predict_proba(X),axis=1)
        return self.unique_[idx]
```



Testing OVR Logistic Regression

```
%%time
ds = load_iris()
X = ds.data
y = ds.target # note problem is NOT binary anymore, there are three classes!

X_train, X_test, y_train, y_test = train_test_split(X,y, train_size = 0.8, test_size=0.2)

lr = LogisticRegression(0.1,500)
lr.fit(X_train,y_train)
print(lr)

yhat = lr.predict(X_test)
print('Accuracy of: ',accuracy_score(y_test,yhat))
```

```
MultiClass Logistic Regression Object with coefficients:
[[ 0.29773533  0.4533672   1.55246277 -2.41143016 -1.13092625]
 [ 0.38589636  0.31560559 -1.28306232  0.5421915  -0.97321777]
 [-0.81705268 -1.49226693 -1.41357874  2.20417964  2.04839556]]
Accuracy of:  0.9666666666666667
CPU times: user 35.5 ms, sys: 3.66 ms, total: 39.2 ms
Wall time: 37.6 ms
```

Ours

`Sklearn` has some efficiency that we do not yet implement:

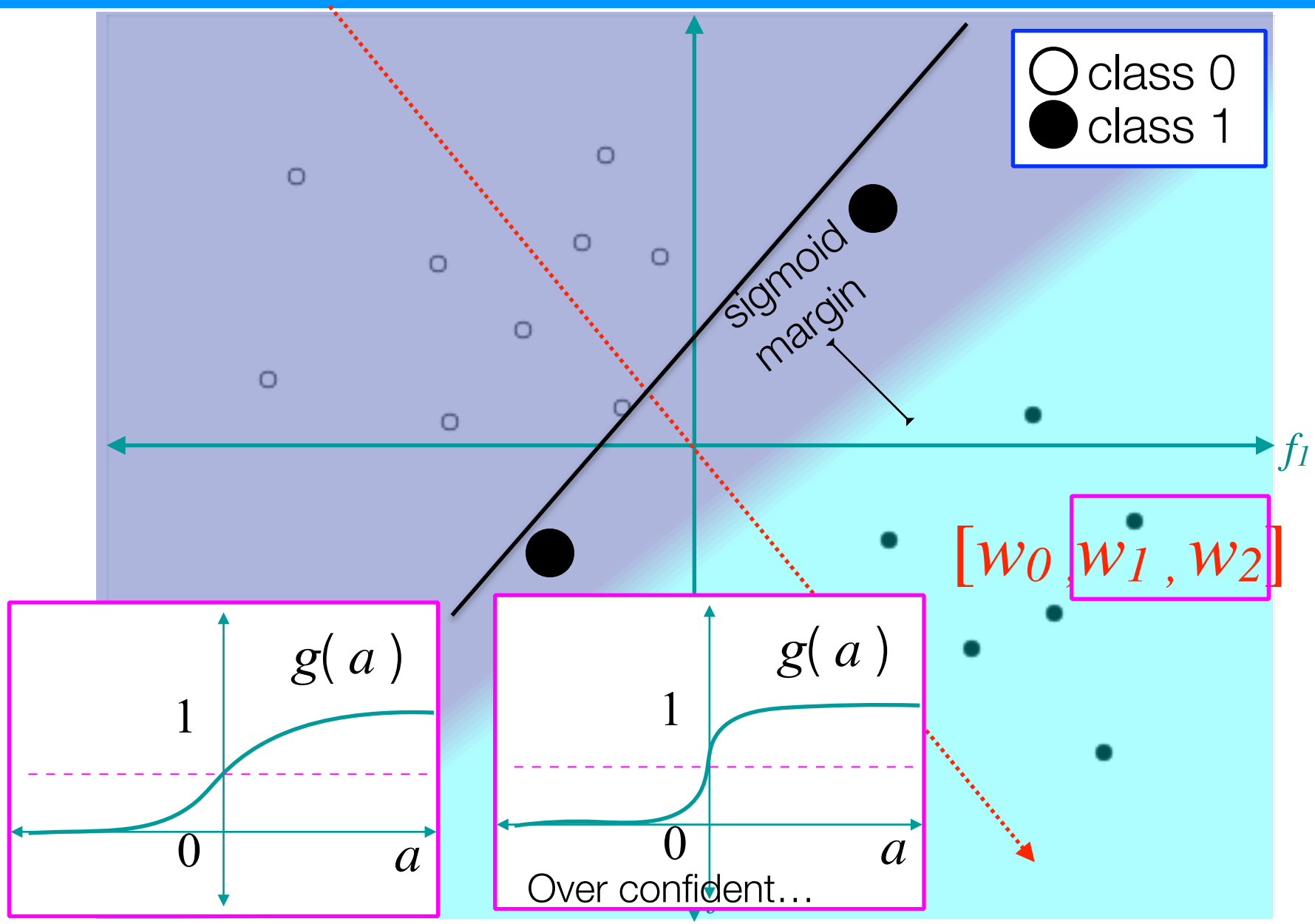
- regularization (try not to overfit)
- better direction finding for the next step in updating $\mathbf{w}$

```
%%time
from sklearn.linear_model import LogisticRegression as SKLogisticRegression

lr_sk = SKLogisticRegression(solver='lbfgs') # all params default

lr_sk.fit(X_train,y_train)
print(np.hstack((lr_sk.intercept_[:,np.newaxis],lr_sk.coef_)))
yhat = lr_sk.predict(X_test)
print('Accuracy of: ',accuracy_score(y_test,yhat))
```

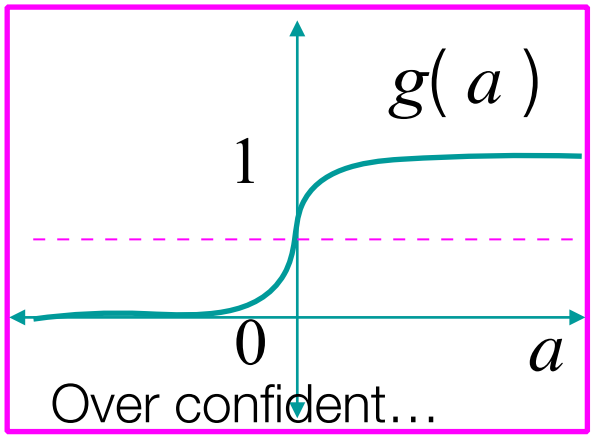
```
[[  9.56665511  -0.4149617    0.85665417  -2.38058111  -0.96355898]
 [  2.11236587   0.49579807  -0.28883991  -0.19632485  -0.92836642]
 [-11.67902098  -0.08083637  -0.56781427   2.57690596   1.8919254 ]]
Accuracy of:  1.0
CPU times: user 17.5 ms, sys: 3.79 ms, total: 21.3 ms
Wall time: 19.5 ms
```

Sklearn

Regularization, Intuition



Regularization, Mathematics



Large values of $\mathbf{w}$ allow sigmoid to be steep

$$l(\mathbf{w})_{reg} = \underbrace{l(\mathbf{w})_{prev}}_{\text{old objective}} - \underbrace{C \cdot \sum_j w_j^2}_{\text{regularization}}$$

$$\underbrace{\left[\left(\sum_{i=1}^{M} (y^{(i)} - g(\mathbf{x}^{(i)} \cdot \mathbf{w}))x_j^{(i)}\right) - C \cdot 2w_j\right]}_{\text{new gradient}}$$

$$\mathbf{w} \leftarrow \mathbf{w} + \eta \left[\underbrace{\nabla l(\mathbf{w})_{prev}}_{\text{previous gradient}} - C \cdot 2\mathbf{w}\right]$$

```
class RegularizedBLR(VectorBLR):
    def __init__(self, C=0.0, **kwds):
        self.C = C
        super().__init__(**kwds) # call parent

    def _get_gradient(self,X,y):
        gradient = super()._get_gradient(X,y)
        gradient[1:] += -2 * self.C * self.w_[1:]
        return gradient
```

C is a hyperparameter controlling how much we want to optimize the previous objective function versus keep the weights small.

Review

Objective Function

$$l(\mathbf{w}) = \sum_i y^{(i)} \ln\left(g(\mathbf{x}^{(i)} \cdot \mathbf{w})\right) + (1 - y^{(i)}) \ln\left(1 - g(\mathbf{x}^{(i)} \cdot \mathbf{w})\right)$$

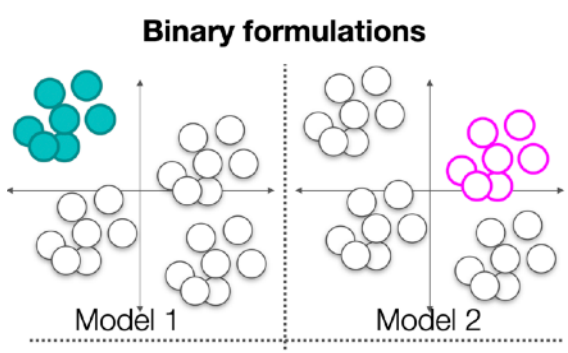
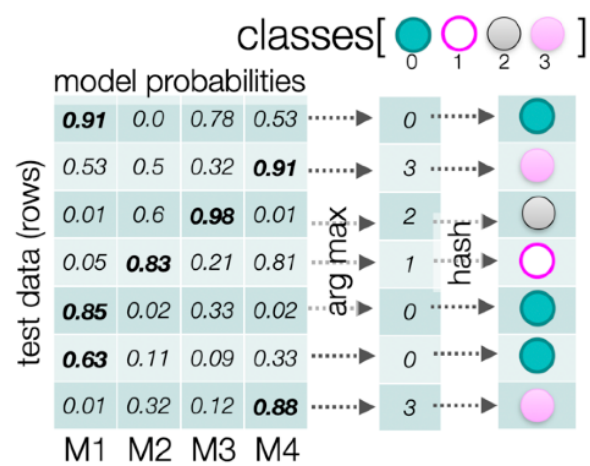
Update Function

$$\underbrace{\mathbf{w}}_{\text{new vect}} \leftarrow \underbrace{\mathbf{w}}_{\text{old vect}} + \frac{\eta}{M} \sum_{i=1}^{M} (y^{(i)} - g(\mathbf{x}^{(i)} \cdot \mathbf{w})) \cdot \mathbf{x}^{(i)}$$

Vectorized Update

$$\mathbf{w} \leftarrow \mathbf{w} + \eta \cdot \text{mean}(\mathbf{y}_{diff} \odot \mathbf{X})_{columns}$$

Regularization

$$l(\mathbf{w})_{reg} = l(\mathbf{w})_{prev} - C \cdot \sum_j w_j^2 \qquad \mathbf{w} \leftarrow \mathbf{w} + \eta \left[\nabla l(\mathbf{w})_{prev} - C \cdot 2\mathbf{w} \right]$$

Multi-class Objective